

Milestone 3 Report

Sports Ball Image Classification Using CNN and Transfer Learning

Done by:

TEAM NAME: WE BARE BEARS

Introduction

Artificial Intelligence (AI) and Deep Learning have become essential technologies in modern computer vision systems. One of the most important applications of deep learning is image classification, where a computer system learns to recognize and classify objects inside images automatically.

This project focuses on building an intelligent image classification system capable of recognizing different types of sports balls using Convolutional Neural Networks (CNNs) and Transfer Learning techniques.

The system was trained to classify six categories of sports balls:

American football

Baseball

Basketball

Football (soccer ball)

Golf ball

Tennis ball

To achieve high accuracy with efficient training time, the project used the pretrained MobileNetV2 CNN architecture provided by TensorFlow/Keres.

Project Objectives

The objectives of this project are:

- Build a CNN-based image classification system
- Train a deep learning model to classify sports balls
- Apply transfer learning to improve performance
- Perform image preprocessing and augmentation
- Evaluate model accuracy and performance
- Save the trained model for future deployment

• Dataset Structure

- The dataset was divided into training and testing folders.
- dataset-balls/

```
|
├── train/
│   ├── american_football/
│   ├── baseball/
│   ├── basketball/
│   ├── football/
│   ├── golf_ball/
│   └── tennis_ball/
└── test/
    ├── american_football/
    ├── baseball/
    ├── basketball/
    ├── football/
    ├── golf_ball/
    └── tennis_ball/
```

Number of Images

Dataset	Images
---------	--------

Training Images	1407
-----------------	------

Testing Images	289
----------------	-----

Development Environment

The project was implemented using Google Colab and Python.

Libraries Used

Library	Purpose
TensorFlow/Keres	Deep learning framework
NumPy	Numerical processing
Matplotlib	Data visualization
Seaborn	Confusion matrix visualization
Scikit-learn	Performance metrics
Google Colab	Cloud-based execution

Methodology

The implementation process consisted of the following stages:

1. Dataset loading
2. Image preprocessing
3. Data augmentation
4. CNN model construction
5. Transfer learning
6. Model training
7. Model evaluation
8. Saving the trained model
9. Image prediction and classification

Data Preprocessing

The dataset images were preprocessed before training.

Image Resizing

All images were resized to:

`224 × 224 pixels`

This size is required by the MobileNetV2 architecture.

Image Normalization

Pixel values originally ranged between:

`0 → 255`

They were normalized to:

`0 → 1`

using:

`rescale = 1. / 255`

Normalization improves:

- training stability
- convergence speed
- model performance

Data Augmentation

To improve generalization and reduce overfitting, data augmentation techniques were applied to the training images.

Augmentation Methods Used

Method	Purpose
Rotation	Handle different object orientations
Zoom	Improve scale robustness
Horizontal Flip	Increase dataset diversity

Augmentation Code

```
train_datagen = ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=20,  
    zoom_range=0.2,  
    horizontal_flip=True  
)
```

8. Convolutional Neural Network (CNN)

The project used a CNN architecture for image classification.

A CNN (Convolutional Neural Network) is a deep learning model specifically designed for image processing and feature extraction.

CNNs automatically learn:

- edges
- shapes
- textures
- object patterns

from images during training.

Transfer Learning

Instead of building a CNN from scratch, transfer learning was used.

MobileNetV2

The pretrained CNN architecture used was:

MobileNetV2

MobileNetV2 was pretrained on the ImageNet dataset containing millions of images.

Advantages of Transfer Learning

Advantage	Benefit
Faster training	Reduced training time
Higher accuracy	Better feature extraction
Lower computation	Efficient model
Less data required	Good for small datasets

Model Architecture

The model architecture consisted of:

1. MobileNetV2 base model
2. Global Average Pooling layer
3. Dense hidden layer
4. Dropout layer
5. Softmax output layer

Model Layers

Layer	Purpose
MobileNetV2	Feature extraction
GlobalAveragePooling2D	Reduce dimensions
Dense(128)	Learning patterns
Dropout(0.3)	Prevent overfitting
Dense(6, softmax)	Final classification

Model Compilation

The model was compiled using:

Parameter	Value
Optimizer	Adam
Learning Rate	0.001
Loss Function	categorical_crossentropy
Metric	accuracy

Training Process

The model was trained for:

5 epochs

with a batch size of:

32

Training Results

The model achieved excellent results during training.

Final Performance

Metric	Result
Training Accuracy	99.0%
Validation Accuracy	97.9%
Validation Loss	0.0437

Epoch Results

Epoch	Training Accuracy	Validation Accuracy
1	86.57%	97.58%
2	97.16%	98.27%
3	99.15%	98.96%
4	99.22%	98.62%
5	99.00%	97.92%

The model converged quickly because transfer learning provides pretrained image features.

Model Evaluation

The model was evaluated using:

- Accuracy
- Validation loss
- Classification report
- Confusion matrix

These metrics help analyze:

- overall performance
- class-level performance
- misclassifications

Classification Prediction

The trained model can classify new images by:

1. Loading the image
2. Resizing it
3. Normalizing pixel values
4. Running prediction
5. Displaying:
 - predicted class
 - confidence score

Example output: Prediction: basketball
Confidence: 98.73%

Conclusion

This project successfully developed a CNN-based image classification system capable of recognizing six different types of sports balls with very high accuracy.

The use of MobileNetV2 transfer learning significantly improved performance while reducing training time and computational requirements.

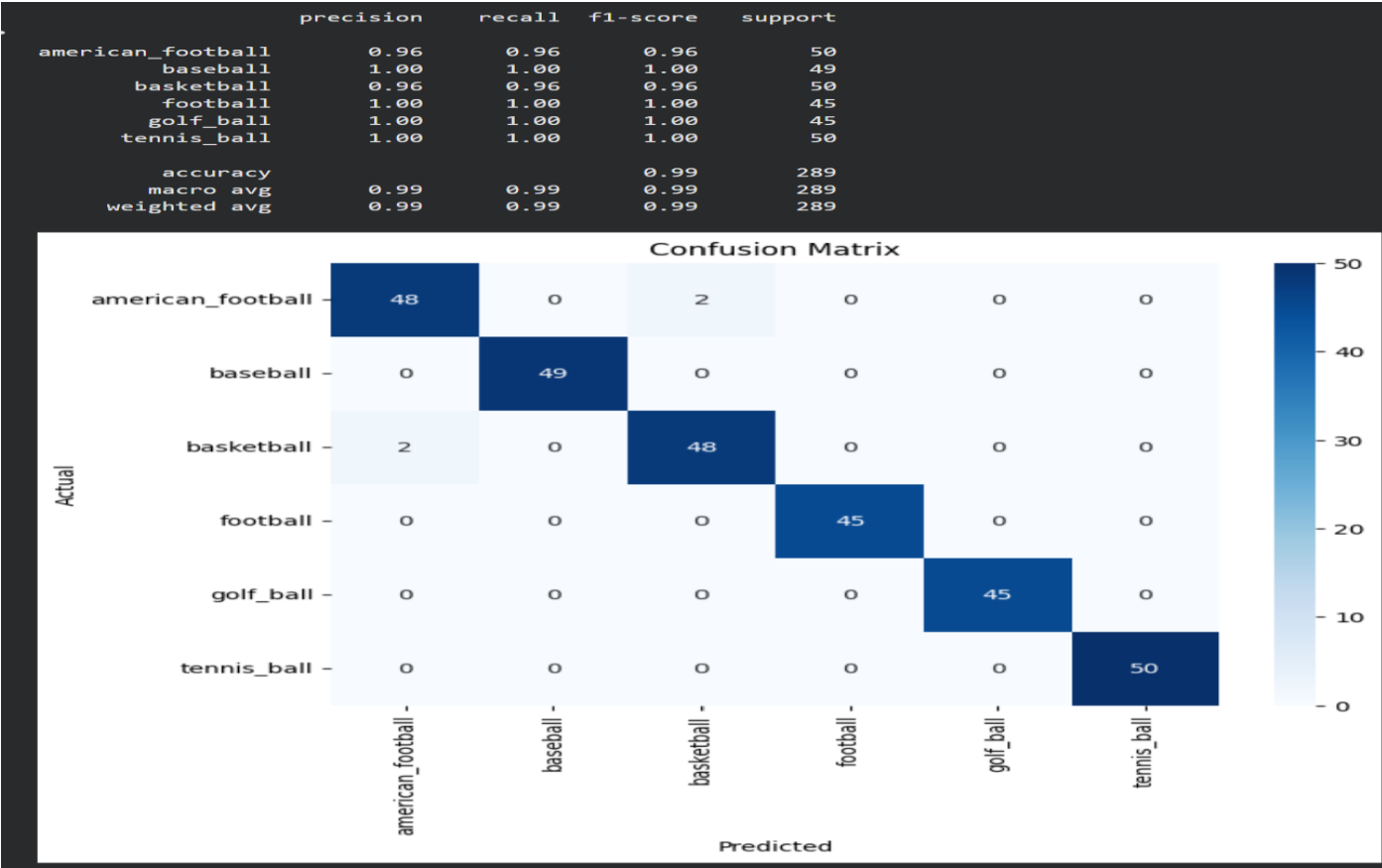
The final model achieved approximately:

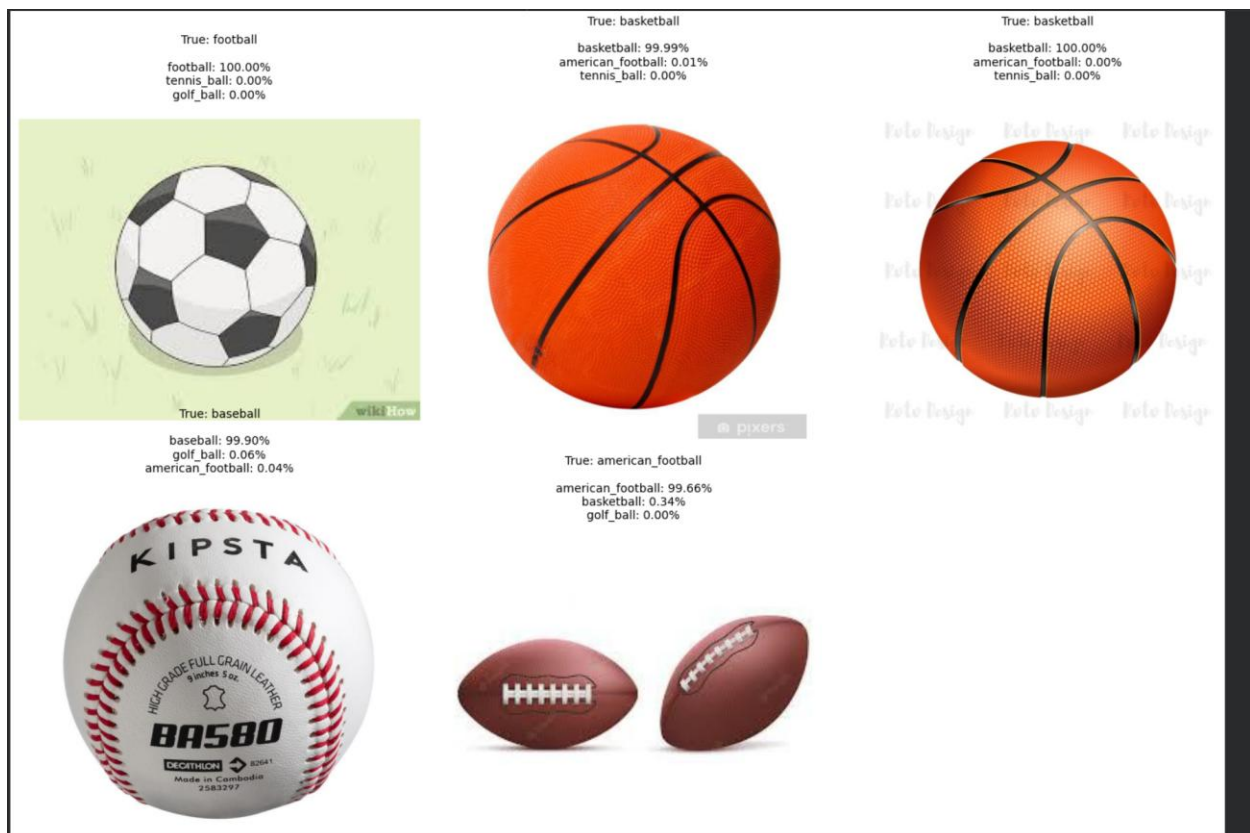
97.9% validation accuracy

which demonstrates the effectiveness of CNNs and transfer learning for image classification tasks.

The project also highlighted the importance of:

- preprocessing
- augmentation
- proper model evaluation
- pretrained CNN architectures





Show top 3 probabilities for some test photos using Milestone 3 code

ANYTHING REGARDING MILESTONE 2 IS in the IPNYB file